

- Download (see the [Links](#) section at the end for source URLs) and extract the tools:

```
bash> cp lcov-1.8.tar.gz unittest-cpp-1.4.zip valgrind-3.5.0.tar.bz2 $TOOLSROOT
bash> cd $TOOLSROOT
bash> tar -xvzf lcov-1.8.tar.gz
bash> unzip unittest-cpp-1.4.zip
bash> tar -xvzf valgrind-3.5.0.tar.bz2
```

- Configure and build *UnitTest++*:

```
bash> cd $TOOLSROOT/UnitTest++/
bash> make
```

- Configure, build and install *lcov*:

```
bash> cd $TOOLSROOT/lcov-1.8
bash> make
bash> sudo make install
```

- Configure, build and install *valgrind*:

```
bash> cd $TOOLSROOT/valgrind-3.5.0
bash> ./configure
bash> make
bash> sudo make install
```

Getting started with unit testing

Here's a snippet of source code that compares two integers. Leave the commented lines as they are; we will uncomment them later in the article.

```
bash> cat $SRCROOT/test.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 int compare_function(int a, int b)
4 {
5     int result = 0;
6     int *p;
7     if ( a > b ) {
8         result = 1;
9     } else if ( a < b ){
10         result = -1;
11     }
12
13     // p = malloc(sizeof(int) * 10);
14     // free(p);
15     return result;
16 }
bash> cat $SRCROOT/test.h
int compare_function(int a, int b);
```

Unit testing generally includes a data generation part that feeds test data to the code that is being tested, and a set of

logically related test cases that are grouped into one or more test suites. The following test program is explained below the code snippet.

```
bash> cat $TESTROOT/testUT.cpp
#include <UnitTest++.h>
#include <TestReporterStdout.h>
#ifdef __cplusplus
extern "C" {
#endif
#include <stdio.h>
#include "test.h"
extern int compare_function(int,int);
#ifdef __cplusplus
}
#endif
```

```
class dataFixture {
public :
    dataFixture() {}
    ~dataFixture() {}
    int getGreaterElt(int a)    { return (a+1) ; }
    int getLesserElt(int a)    { return (a-1) ; }
};
```

```
SUITE(TestUTSuite)
{
    TEST(TestUTCompareGreater)
    {
        int result;
        result = compare_function(2,3);
        CHECK(result == -1);
    }

    TEST_FIXTURE(dataFixture, TestUTCompareGreaterFixture)
    {
        int result;
        result = compare_function(2,getGreaterElt(2));
        CHECK_EQUAL(result,-1);
    }

    TEST(TestUTCompareEqual)
    {
        int result;
        result = compare_function(2,2);
        CHECK_EQUAL(result,0);
    }

    TEST_FIXTURE(dataFixture, TestUTCompareLesser)
    {
        int result;
        result = compare_function(2,getLesserElt(2));
        CHECK_EQUAL(result,1);
    }
}
```